SVD GPU benchmark: Pavel-s_SVD on NVIDIA GB10

30 October 2026 · PyTorch 2.14.1+cu130 · CUDA 13.0 · float32 square Gaussian matrices, n = 32, 64, 128, 256 · interactive version: report.html (claude.ai Dashboard runtime)

Findings

- **Speed.** Every repository solver is limited by kernel launches and host synchronization, not by arithmetic: at most about 6 GFLOP/s against 49 GFLOP/s for cuSOLVER. Time grows roughly linearly in n instead of as n^3. At n = 256 the fastest repository SVDs (polar_polynomial, polar_newton_schulz, polar_cans, about 0.69 s) are about 97× slower than torch.linalg.svd (7.2 ms).
- **Slowest.** bidiag_qr / recipes.golub_kahan / recipes.lanczos (bdsqr is a Python loop with Givens rotations) and block_jacobi (4960 inner Jacobi solves at n = 128) are an order of magnitude slower; their n = 256 cases were skipped.
- **Accuracy.** bidiag_dc reconstructs best (6e-7, float64 merges), then bidiag_qr (2e-6); the Jacobi family is at about 5e-5 and the polar family at about 1e-4.
- **Gram methods.** gram_jacobi and gram_dc square the condition number and lose the smallest singular values at n = 256 (relative σ error 1.0).
- **Failures.** None. The standard svdlab check on CUDA passes for my_svd, torch and jacobi.

Median time per SVD, ms (float32)

Reference (cuSOLVER) · Golub-Kahan bidiagonal · One-sided Jacobi · QR then Jacobi · Gram eigensolver · Polar decomposition · Randomized / Krylov · Lanczos bidiagonal

method	n=32	64	128	256	GFLOP at largest n	GFLOP/s	largest n
torch.linalg.svd	0.231	1.06	2.67	7.16	0.352	49.2	256
bidiag_dc	90.9	190	391	857	0.573	0.7	256
bidiag_qr	597	2,272	8,712	skipped	skipped	skipped	256
recipes.golub_kahan	600	2,278	8,761	skipped	skipped	skipped	256
block_jacobi	1,128	5,875	31,828	skipped	skipped	skipped	256
jacobi	71.6	163	403	890	1.805	2.0	256
qr	77.7	175	426	938	1.894	2.0	256
recipes.qr_then_svd	78.1	193	464	935	1.894	2.0	256
tsqr	77.8	175	426	935	1.894	2.0	256
gram_dc	83.0	174	357	732	0.285	0.4	256
gram_jacobi	64.8	150	416	769	1.740	2.3	256
polar_cans	67.7	153	344	694	4.457	6.4	256
polar_halley	96.3	220	496	1,057	2.511	2.4	256
polar_newton_schulz	66.2	151	343	692	3.517	5.1	256
polar_polynomial	66.1	151	342	691	3.551	5.1	256
polar_qdwh	89.6	199	440	883	2.304	2.6	256
block_krylov	90.2	200	442	1,027	2.141	2.1	256
randomized	103	225	528	1,127	2.320	2.1	256
recipes.randomized	103	225	491	1,134	2.320	2.0	256
subspace	103	225	527	1,127	2.320	2.1	256
lanczos	77.2	156	426	927	2.140	2.3	256
recipes.lanczos	594	2,261	8,600	skipped	skipped	skipped	256

GFLOP are executed operations (torch.linalg.svd: textbook estimate $21n^3$). “skipped”: not run at n = 256 because n = 128 took over 5 s.

How each SVD works

All repository solvers first scale A by max $|a_{ij}|$ and factor a tall matrix (a wide one is transposed). Dashed boxes are iterative steps; the counters reported for them are in the All runs table.

Golub–Kahan bidiagonal

A → **Householder bidiagonalization**
$A = Q\,B\,P^T$, B upper bidiagonal (d, e) →

↻ iterative
Bidiagonal SVD
bidiag_qr: implicit-shift QR (bdsqr), Givens bulge chasing
bidiag_dc: divide and conquer, secular equation in float64 →

Back-transform
$U = Q\,U_B$, $V = P\,V_B$

bidiag_qr, bidiag_dc, golub_kahan (recipe = bidiag_qr through svdlab.blocks)

One-sided Jacobi (Hestenes)

A →

↻ iterative
Sweeps of plane rotations
$W = A\,V$; rotate column pairs (p, q) so $w_p^T w_q = 0$
round-robin: n/2 disjoint pairs per round, n−1 rounds per sweep; stop when max cos ≤ 10ε

→ **Normalize**
$\sigma_j = \|w_j\|$, $U = W\,\Sigma^{-1}$

jacobi; block_jacobi rotates blocks of 4 columns, solving each 8-column pair by inner Jacobi

QR then Jacobi

A → **Householder QR**
$A = Q\,R$ (tsqr: tiled reduction tree; one tile for square A)

→ ↻ iterative
Jacobi SVD of R
$R = U_R\,\Sigma\,V^T$, n × n →

Back-transform
$U = Q\,U_R$

qr, tsqr, qr_then_svd (recipe)

Gram matrix eigensolver

A → **Gram matrix**
$G = A^T A$ (one GEMM; squares the condition number) →

↻ iterative
Symmetric eigensolver
gram_jacobi: two-sided Jacobi
gram_dc: tridiagonalize, then Cuppen divide and conquer →

Recover U
$\sigma = \sqrt{\lambda}$, $U = A\,V\,\Sigma^{-1}$

gram_jacobi, gram_dc

Polar decomposition

A →

↻ iterative
Polar iteration
$X_0 = A/\|A\|_F \to U_p$
Newton–Schulz $X(3I - X^TX)/2$; polynomial (5th order); CANS (Remez minimax); Halley; QDWH (Cholesky or QR)

→ **Hermitian factor**
$H = U_p^T A$ → ↻ iterative
Jacobi eigensolver
$H = V\,\Sigma\,V^T$ →

Recover U
$U = U_p\,V$

polar_qdwh, polar_newton_schulz, polar_halley, polar_polynomial, polar_cans

Randomized range finder

A → **Sketch**
$Y = A\,\Omega$, Ω Gaussian n × (k+8) →

↻ iterative
Power iterations (2)
$Q = qr(Y)$; $Z = qr(A^TQ)$; $Y = A\,Z$
block_krylov keeps every block →

↻ iterative
Small SVD
$Q^TA = U_B\,\Sigma\,V^T$ by Jacobi →

Lift
$U = Q\,U_B$

randomized, subspace, block_krylov, randomized (recipe). Run here with k = n, so the result is a full SVD

Golub–Kahan–Lanczos

A →

↻ iterative

Lanczos bidiagonalization

k steps of A v and $A^T u$ with full re-orthogonalization (two passes)

→

↻ iterative

Small SVD

blocks: Jacobi on $U_k^T A V_k$

recipe: bdsqr on the Lanczos bidiagonal

→

Lift

$U = U_k U_B$, $V = V_k V_B$

lanczos, lanczos (recipe). Run here with k = n steps

Reference: torch.linalg.svd

A →

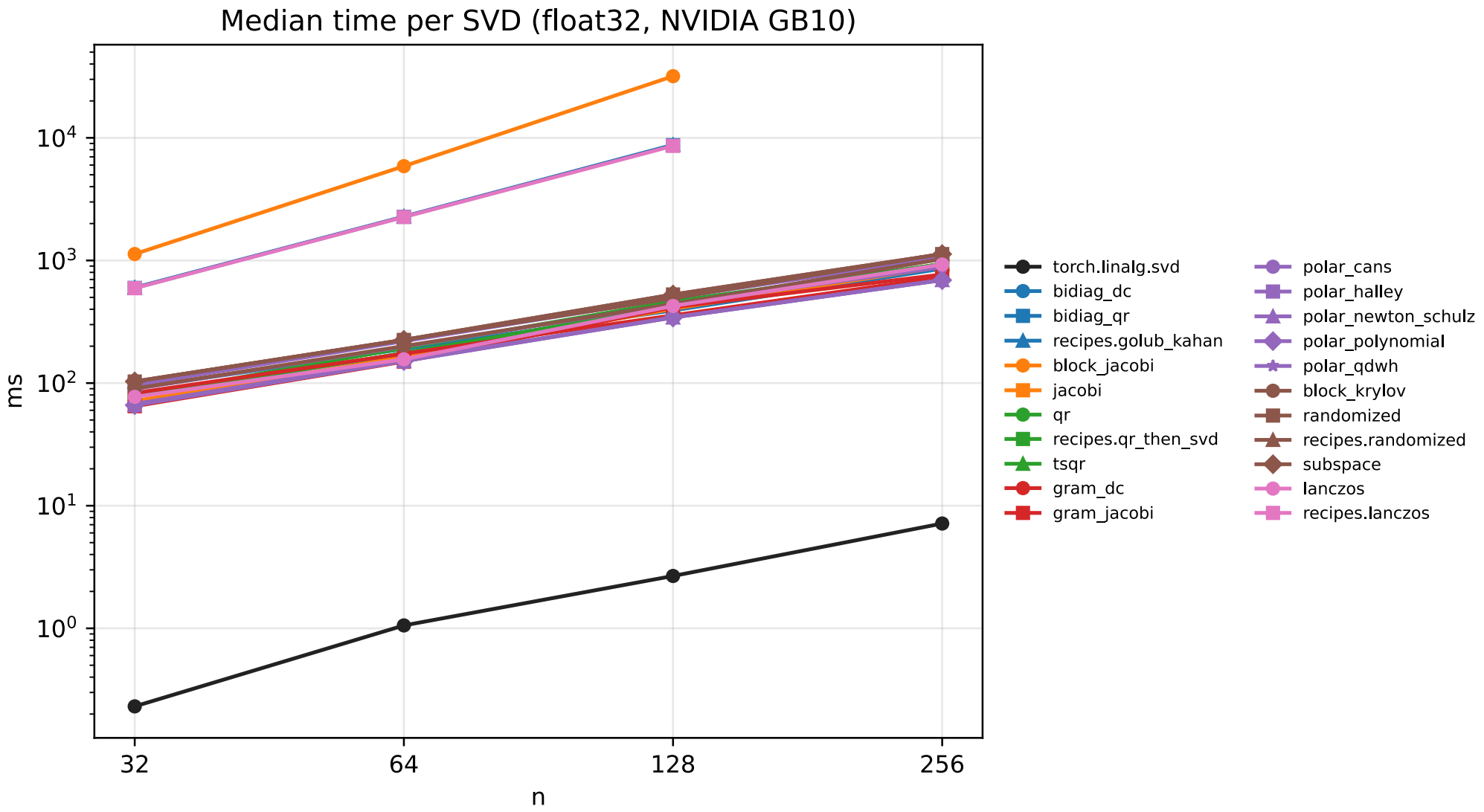
cuSOLVER gesvd family

blocked Householder bidiagonalization and bidiagonal QR inside vendor kernels

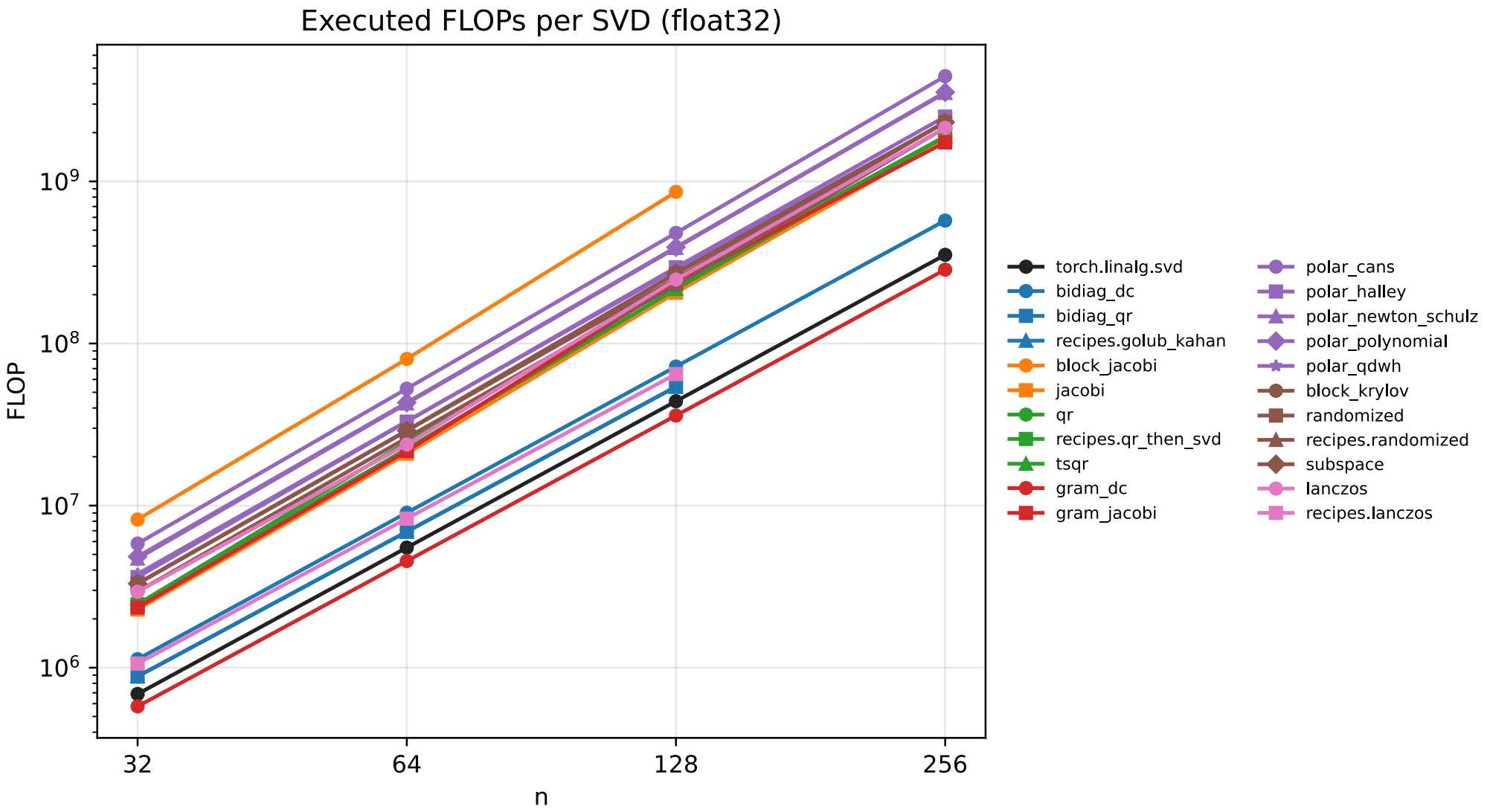
→ $U \Sigma V^T$

FLOPs for this one are the textbook estimate $21n^3$, not a count

Median time vs n

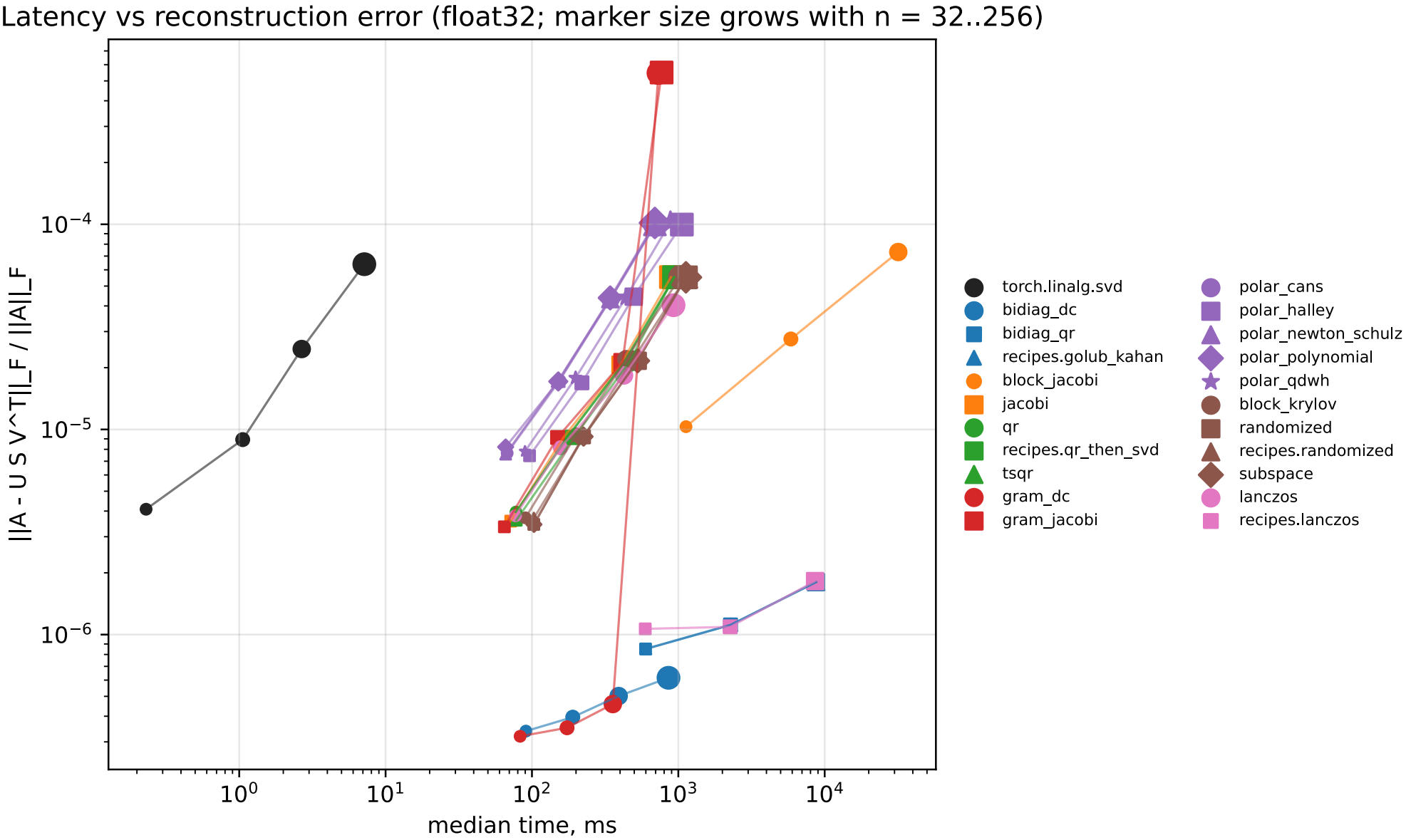


Executed FLOPs vs n



Latency vs reconstruction error

Each method's points are joined from n = 32 to 256; marker size grows with n.



Accuracy and iterations at n = 128 (float32)

method	‖A−UΣVᵀ‖/‖A‖	‖UᵀU−I‖	‖VVᵀ−I‖	max rel. σ error	iterations
torch.linalg.svd	2.5e-05	1.7e-04	3.8e-04	2.2e-05	vendor kernel
bidiag_dc	5.0e-07	5.1e-06	5.2e-06	6.9e-06	Jacobi 155 sweeps in 32 calls (max 5)
bidiag_qr	1.8e-06	1.4e-05	1.4e-05	7.2e-06	bdsqr 163 it
recipes.golub_kahan	1.8e-06	1.4e-05	1.4e-05	7.2e-06	bdsqr 163 it
block_jacobi	7.3e-05	4.0e-05	9.1e-04	4.7e-05	outer 10 sweeps, Jacobi 14803 sweeps in 4960 calls (max 6)
jacobi	2.1e-05	3.9e-05	2.4e-04	1.3e-05	Jacobi 11 sweeps
qr	2.1e-05	3.9e-05	2.5e-04	1.6e-05	Jacobi 11 sweeps
recipes.qr_then_svd	2.1e-05	4.0e-05	2.5e-04	1.4e-05	Jacobi 12 sweeps
tsqr	2.1e-05	3.9e-05	2.5e-04	1.6e-05	Jacobi 11 sweeps
gram_dc	4.6e-07	1.3e-03	5.2e-06	4.1e-04	Jacobi-eigh 158 sweeps in 32 calls (max 5)
gram_jacobi	2.1e-05	3.6e-02	2.5e-04	2.6e-04	Jacobi-eigh 11 sweeps
polar_cans	4.3e-05	2.5e-04	2.5e-04	3.0e-05	polar 22 it, Jacobi-eigh 9 sweeps
polar_halley	4.4e-05	2.5e-04	2.5e-04	3.0e-05	polar 10 it, Jacobi-eigh 9 sweeps
polar_newton_schulz	4.3e-05	2.5e-04	2.5e-04	3.1e-05	polar 22 it, Jacobi-eigh 9 sweeps
polar_polynomial	4.4e-05	2.5e-04	2.5e-04	4.2e-05	polar 15 it, Jacobi-eigh 9 sweeps
polar_qdwh	4.4e-05	2.5e-04	2.5e-04	3.5e-05	polar 5 it, Jacobi-eigh 9 sweeps
block_krylov	2.2e-05	3.9e-05	2.5e-04	1.4e-05	power 2 it, Jacobi 10 sweeps
randomized	2.2e-05	4.0e-05	2.5e-04	1.4e-05	power 2 it, Jacobi 11 sweeps
recipes.randomized	2.2e-05	4.0e-05	2.6e-04	1.4e-05	Jacobi 10 sweeps
subspace	2.2e-05	4.0e-05	2.5e-04	1.4e-05	power 2 it, Jacobi 11 sweeps
lanczos	1.8e-05	4.0e-05	2.3e-04	1.2e-05	Lanczos 128 steps, Jacobi 11 sweeps
recipes.lanczos	1.8e-06	1.3e-05	1.3e-05	1.7e-06	bdsqr 160 it

float64 subset, median ms

method	n=32	64	128	256	recon. error at 128
torch.linalg.svd	3.42	11.0	30.7	91.8	3.4e-14
bidiag_qr	784	2,881	11,502	skipped	4.3e-15
jacobi	71.2	179	402	969	2.2e-14
polar_qdwh	106	232	506	1,102	4.2e-14

Algorithms

method	steps	iterations	rank
torch.linalg.svd	cuSOLVER gesvd-family (vendor library) - reference	internal	full
bidiag_qr	Householder bidiagonalization (A = Q B P^H) -> implicit-shift bidiagonal QR (bdsqr, Givens chasing, lock-step batched) -> U = Q Ub, V = P Vb	yes: bdsqr QR iterations until superdiagonal deflates (tol 10*eps, max_iter 60n); count = bdsqr_iterations	full (truncate afterwards)
bidiag_dc	Householder bidiagonalization -> recursive divide-and-conquer bidiagonal SVD in float64 (split, recurse, rank-one secular-equation merge by bisection; leaves n<=4 and hard merges solved by one-sided Jacobi)	yes: bisection inside each merge + Jacobi on leaves/refinements (max_sweeps 80); recursion depth ~log2(n/4)	full
jacobi	one-sided Jacobi on A directly, round-robin cyclic sweeps; s = column norms, U = W/s	yes: sweeps until off <= 10*eps (max_sweeps 60); count = jacobi_sweeps	full
block_jacobi	block one-sided Jacobi: columns grouped in blocks of 4, round-robin over block pairs, each pair (8 columns) orthogonalized by inner one-sided Jacobi SVD; convergence test on full Gram correlation	yes: outer block sweeps (max 30, tol 40*eps) x inner Jacobi sweeps per block pair	full
qr	Householder QR (A = QR) -> one-sided Jacobi SVD of R (n x n) -> U = Q Ur	yes: Jacobi sweeps on R (max 60)	full
tsqr	tall-skinny QR (row tiles of 4n rows, binary reduction tree; for square/small m it is a single Householder QR) -> one-sided Jacobi SVD of R -> U = Q Ur	yes: Jacobi sweeps on R	full
gram_jacobi	G = A^H A -> two-sided Jacobi eigendecomposition of G -> s = sqrt(eig), U = A V / s (null columns completed)	yes: Hermitian Jacobi sweeps (max 60); count = eigen_sweeps	full (values below eps*n*lambda_max discarded/zeroed; squares the condition number)
gram_dc	G = A^H A -> Householder tridiagonalization -> Cuppen divide-and-conquer tridiagonal eigensolver (float64 merges, leaves n<=4 by Jacobi eigh) -> s = sqrt(eig), U = A V / s	yes: bisection in merges + leaf Jacobi	full
polar_qdwh	normalize by ||A||_F -> QDWH polar iteration (dynamically weighted Halley; Cholesky + two triangular solves per step, or QR of [sqrt(c)X; I] when c>100) -> H = Up^H A -> Jacobi eigh of H -> U = Up V	yes: polar iterations until ||X_{k+1}-X_k||_max <= 10*eps (max 40, typically ~6) + Jacobi eigh sweeps	full
polar_newton_schulz	normalize by ||A||_F -> Newton-Schulz polar iteration X <- X(3I - X^H X)/2 (GEMM only) -> H = Up^H A -> Jacobi eigh -> U = Up V	yes: polar iterations (max 120, tol 10*eps); slow for small sigma_min/||A||_F + eigen sweeps	full
polar_halley	normalize -> Halley polar iteration (a,b,c)=(3,1,3), Cholesky-based inverse per step -> H = Up^H A -> Jacobi eigh -> U = Up V	yes: polar iterations (max 40) + eigen sweeps	full
polar_polynomial	normalize -> 5th-order polynomial polar iteration X(15I - 10G + 3G^2)/8, G = X^H X (GEMM only) -> H -> Jacobi eigh -> U	yes: polar iterations (max 120) + eigen sweeps	full
polar_cans	normalize -> CANS polar iteration: odd minimax polynomial of order 3 (Remez coefficients recomputed every step from bounds [1-e,1+e]) applied by GEMMs -> H -> Jacobi eigh -> U	yes: polar iterations (max 120) + Remez + eigen sweeps	full

method	steps	iterations	rank
randomized	Halko-Martinsson-Tropp: Omega Gaussian (n x l, l = min(rank+8, n)), Q = qr(A Omega), 2 power iterations (QR each half step) -> Jacobi SVD of Q^H A -> U = Q Ub	fixed 2 power iterations + Jacobi sweeps on the small matrix	truncated (rank parameter); with rank=n it is a full SVD of Q^H A
subspace	same as randomized (subspace iteration with QR re-orthonormalization)	fixed iterations=2 + Jacobi sweeps	truncated
block_krylov	randomized block Krylov: keep all power-iteration blocks [Q0, Q1, Q2] (each reorthogonalized twice against previous), QR of concatenation -> Jacobi SVD of Q^H A	fixed iterations=2 (stops early when Krylov basis fills m columns) + Jacobi sweeps	truncated
lanczos	Golub-Kahan-Lanczos bidiagonalization with full reorthogonalization (steps = min(m,n, 2*rank+8)) -> recompute core Ub^H A Vb -> Jacobi SVD of the core	fixed `steps` Lanczos steps + Jacobi sweeps on core	truncated (full when steps = n)
recipes.golub_kahan	blocks.bidiagonalize (Householder) -> blocks.bidiagonal_svd (bdsqr) -> U = Q Ub, V = P Vb; same pipeline as bidiag_qr but through the recipe API	yes: bdsqr iterations	full
recipes.qr_then_svd	blocks.qr (Householder) -> blocks.jacobi_svd(R) -> U = Q Ur	yes: Jacobi sweeps on R	full
recipes.randomized	HMT via blocks: sketch A Omega (rank+8 cols), 2 power iterations with blocks.qr, blocks.jacobi_svd(Q^H A)	fixed power_iterations=2 + Jacobi sweeps	truncated (rank required)
recipes.lanczos	blocks.lanczos_bidiagonalize (GKL, full reorth, steps = min(m,n,2*rank+8)) -> blocks.bidiagonal_svd (bdsqr on the Lanczos bidiagonal) -> U = U_L Ub	fixed steps + bdsqr iterations	truncated (rank required)
recipes.jacobi	identical to blocks.svd:jacobi (blocks.jacobi_svd) - not benchmarked separately	Jacobi sweeps	full

Standard check: svdlab check on CUDA

method	case	shape / dtype	ms	recon	σ rel	orth	result	GPU check
my_svd	gaussian_tall	200x120 float64	459	3.7e-15	3.5e-15	3.8e-15	ok	PASS
my_svd	gaussian_wide	100x180 float64	310	3.3e-15	3.8e-15	3.4e-15	ok	PASS
my_svd	ill_conditioned_1e8	160x120 float64	1,017	3.9e-15	1.9e-09	6.4e-15	ok	PASS
my_svd	clustered	160x120 float64	705	4.7e-15	2.9e-14	4.9e-15	ok	PASS
my_svd	sparse_dense_storage	200x150 float64	497	4.5e-15	5.3e-15	4.5e-15	ok	PASS
my_svd	complex_gaussian	120x80 complex128	300	3.1e-15	2.6e-15	3.1e-15	ok	PASS
my_svd	low_rank_k10	300x200 float64	765	8.7e-06	1.8e-15	1.9e-15	ok	PASS
my_svd	low_rank_k16_fp32	400x300 float32	1,150	8.7e-04	1.5e-05	2.8e-05	ok	PASS
torch	gaussian_tall	200x120 float64	30.4	3.1e-14	2.9e-14	3.0e-14	ok	PASS
torch	gaussian_wide	100x180 float64	28.2	3.0e-14	2.5e-14	2.8e-14	ok	PASS
torch	ill_conditioned_1e8	160x120 float64	37.3	5.1e-14	3.5e-10	3.8e-14	ok	PASS
torch	clustered	160x120 float64	37.8	3.9e-14	3.6e-14	5.2e-14	ok	PASS
torch	sparse_dense_storage	200x150 float64	44.3	4.3e-14	3.8e-14	4.2e-14	ok	PASS
torch	complex_gaussian	120x80 complex128	57.0	7.1e-14	5.1e-14	4.8e-14	ok	PASS
torch	low_rank_k10	300x200 float64	176	8.7e-06	2.6e-13	5.0e-13	ok	PASS
torch	low_rank_k16_fp32	400x300 float32	15.6	8.7e-04	5.7e-05	1.4e-04	ok	PASS
jacobi	gaussian_tall	200x120 float64	468	2.0e-14	1.4e-14	2.0e-14	ok	PASS
jacobi	gaussian_wide	100x180 float64	316	1.6e-14	1.1e-14	1.5e-14	ok	PASS
jacobi	ill_conditioned_1e8	160x120 float64	1,081	4.2e-14	1.0e-09	3.5e-14	ok	PASS
jacobi	clustered	160x120 float64	723	4.8e-14	6.1e-14	4.4e-14	ok	PASS
jacobi	sparse_dense_storage	200x150 float64	506	2.5e-14	3.0e-14	2.5e-14	ok	PASS
jacobi	complex_gaussian	120x80 complex128	327	1.3e-14	1.0e-14	1.2e-14	ok	PASS
jacobi	low_rank_k10	300x200 float64	799	8.7e-06	1.9e-14	3.5e-14	ok	PASS
jacobi	low_rank_k16_fp32	400x300 float32	1,203	8.7e-04	7.9e-06	1.4e-05	ok	PASS

How this was measured

- Hardware and software: NVIDIA GB10 (CUDA capability 12.1), PyTorch 2.14.1+cu130, CUDA 13.0, repository Pavel-s_SVD at main on 30 October 2026. The GPU was shared with another, idle process.
- Matrices: one square Gaussian matrix per size, drawn in float64 with seed 1000 + n and cast to float32 (or kept in float64). Every method gets rank = n, so truncated methods also return a full SVD.
- Timing: 3 warm-up runs, then 20 timed runs measured with CUDA events after synchronizing; median and minimum reported. A method whose first run took over 2 s got 1 warm-up and 3 timed runs. Methods slower than 5 s at n = 128 were not run at n = 256.
- FLOPs: matmul FLOPs measured with torch.utils.flop_counter. The repository's Householder, Cholesky, triangular-solve, Jacobi and bdsqr primitives use element-wise kernels, not matmul. Their FLOPs are counted exactly from each call's shape and its actual sweep or iteration count, including masked work. Not counted: float64 secular-equation solves in the divide-and-conquer methods, norms and sorting. torch.linalg.svd uses $21n^3$.
- Accuracy is computed in float64 from the returned factors.
- Raw files: benchmarks/gpu_flops/results/2026-10-30-gb10 (results.json, flops.json, algorithms.json, scorecard output); scripts in benchmarks/gpu_flops.